

Experiment 1 Protocol: Property Recovery from Text-Serialized Structured Objects

Internal Reference Document

June 2026

Contents

1	Overview	3
2	Models	3
3	Domain 1: Geometry	4
3.1	Overview	4
3.2	Object Specification	4
3.3	Serialization Format	5
3.4	Properties	5
3.5	Prompt Template	6
4	Domain 2: Graphs	7
4.1	Overview	7
4.2	Object Specification	7
4.3	Serialization Format	8
4.4	Properties	8
4.5	Prompt Template	9
5	Domain 3: Tabular Data	9
5.1	Overview	9
5.2	Object Specification	10
5.3	Serialization Format	11
5.4	Properties	11
5.5	Prompt Template	12
6	Evaluation	12
6.1	Answer Parsing	12
6.2	Correctness Criteria	13
6.3	Metrics to Report	14
6.4	Statistical Reporting	15
7	Qualitative Failure Analysis	15
8	Deliverables	15

9	Decision Rules	16
10	Execution Checklist	17

1 Overview

This document specifies Experiment 1 (Property Recovery) of the serialization bottleneck project. The experiment measures how accurately language models can recover implicit properties from text-serialized structured objects across three domains: geometry, graphs, and tabular data.

The experiment produces, for each model, a per-property, per-domain, per-complexity accuracy matrix. This matrix is the foundation for all subsequent experiments (pipeline compounding, cross-domain comparison, and mitigation).

Relation to the pilot. A pilot version of this experiment was completed for the geometry domain using Llama-3.1-70B-Instruct on 90 polygons. The pilot validated the experimental harness and confirmed a large implicit-structure bottleneck (Outcome A: strong failure on global properties). This full experiment extends the pilot in three ways: (a) three domains instead of one, (b) six models instead of one, (c) 300 objects per domain instead of 90. The geometry harness built for the pilot should be reused and extended; it does not need to be rewritten.

Execution order. Run DeepSeek-V4-Flash on all three domains first. This is the debugging run. Fix all harness issues, parser edge cases, and prompt problems against this one model before running the other five. Once the infrastructure is validated, run the remaining five models without modifying the harness, prompts, or evaluation logic.

Total scale. 300 objects per domain $\times$ 3 domains $\times$ 6–10 properties per object = approximately 7,200 queries per model. Six models (five full runs, one at 20% coverage) yields roughly 36,000–43,000 queries total.

2 Models

Table 1: Models and their roles in Experiment 1.

Model	Parameters	Access	Coverage	Role
DeepSeek-V4-Flash	284B MoE (13B active)	API	100%	Primary; debug first
Qwen3-32B	32B dense	API (OpenRouter)	100%	Primary
Llama 4 Scout	109B MoE (17B active)	API (OpenRouter)	100%	Primary
Gemini 2.5 Flash-Lite	undisclosed	API (Google)	100%	Small baseline
GPT-4.1 Mini	undisclosed	API (OpenAI)	100%	Closed-source
DeepSeek-V4-Pro	1.6T MoE (49B active)	API	20%	Thinking-mode

Inference parameters (all models except DeepSeek-V4-Pro).

- Temperature: 0 (greedy decoding).
- Max output tokens: 512.
- System prompt: none.
- Prompting regime: zero-shot, direct answer. No few-shot examples, no chain-of-thought instructions.

Inference parameters (DeepSeek-V4-Pro, thinking mode).

- Use the provider’s default thinking-mode configuration.
- Max output tokens: 2,048 (thinking traces can be long).
- Run on a 20% random subsample of all objects, stratified by tier and shape/type category so that the subsample preserves the distributional balance of the full dataset.
- All other settings match the non-thinking models.

API access. All models are accessed through API endpoints. Use retry logic with exponential backoff (3 retries on timeout or rate-limit errors, initial delay 2 seconds, multiplier 2). Record the timestamp, model identifier, and provider for every call.

3 Domain 1: Geometry

3.1 Overview

This domain reuses and extends the pilot infrastructure. The pilot used 90 polygons with 7 properties. The full experiment uses 300 polygons with 9 properties.

3.2 Object Specification

Tier structure. 300 polygons divided into three complexity tiers of 100 polygons each.

Table 2: Geometry dataset tiers.

Tier	Count	Vertices	Coordinates	Purpose
Simple	100	3–8	Integers in $[0, 100]$	Baseline
Medium	100	10–20	Integers in $[0, 100]$	Core measurement
Hard	100	20–40	Floats (2 decimal places) in $[0, 1000]$	Stress test

Shape categories. Within each tier, polygons are split into three shape categories. The split does not need to be exactly equal but should be approximately balanced (30–35 per category per tier).

Table 3: Shape categories.

Category	Description
Convex	All interior angles less than 180° .
Mildly concave	At least one reflex angle, no extreme concavities.
Irregular/elongated	High aspect ratio or non-uniform vertex spacing.

Generation methods. Use the same generation methods as the pilot protocol:

- **Convex:** convex hull of m random points, accept if vertex count falls within the tier’s range.
- **Mildly concave:** radial (star-shaped) generation with controlled $r_{\min}/r_{\max}$ ratio between 0.4 and 0.8.
- **Irregular/elongated:** radial generation followed by affine stretch ($s \in [2.0, 5.0]$ on one axis) and random rotation.

Validity constraints. Every polygon must satisfy all of the following. These are unchanged from the pilot.

1. `polygon.is_valid == True` (Shapely validation).
2. `polygon.is_simple == True` (no self-intersections).
3. `polygon.area > 10` (no degenerate slivers).
4. No two adjacent vertices are identical after rounding.
5. No three consecutive vertices are collinear.
6. Vertex count (excluding closing coordinate) within tier range.
7. All coordinates within tier bounds.

Polygons failing any constraint are rejected and regenerated. Fix and record the random seed.

Orientation randomization. After computing all non-orientation properties, reverse the coordinate order for a random 50% of polygons (same procedure as the pilot). Then compute the orientation ground truth. This ensures roughly 50/50 clockwise/counter-clockwise balance.

3.3 Serialization Format

WKT (Well-Known Text):

```
POLYGON((x1 y1, x2 y2, ..., xn yn, x1 y1))
```

3.4 Properties

Nine properties, classified as local or global.

New properties relative to pilot. The pilot measured 7 properties. This experiment adds `aspect_ratio` and `edge_length_variance`. Aspect ratio is a simple derived property that tests whether the model can compose two local readings (bbox width and height). Edge length variance requires computing all n edge lengths and their variance, which is a global aggregation over local measurements. These two additions fill a gap between the purely local (bbox) and purely global (area, convexity) extremes.

Table 4: Geometry properties.

Property	Type	Locality	Ground Truth	Eval
vertex_count	integer	local	<code>len(poly.exterior.coords) - 1</code>	exact
bbox	4-tuple	local	<code>poly.bounds</code>	rel. error
centroid	2-tuple	global	<code>poly.centroid.coords[0]</code>	normalized ℓ_2
area	float	global	<code>poly.area</code>	rel. error
perimeter	float	global	<code>poly.length</code>	rel. error
convexity	boolean	global	<code>poly.normalize().equals(poly.convex_hull.normalize())</code>	exact
orientation	string	global	sign of shoelace signed area	exact
aspect_ratio	float	global	bbox width / bbox height	rel. error
edge_length_variance	float	global	variance of all edge lengths	rel. error

3.5 Prompt Template

Use the same template structure as the pilot. One query per property per polygon.

You are given a polygon in WKT format.

Polygon:
{wkt_string}

Question:
{question_text}

Respond with only a JSON object in this format:
{"answer": <your answer>}

Table 5: Geometry question text.

Property	Question Text	Answer Example
vertex_count	How many vertices does this polygon have? Do not count the repeated closing coordinate.	{"answer": 6}
bbox	What is the axis-aligned bounding box of this polygon? Give [min_x, min_y, max_x, max_y].	{"answer": [0,0,4,3]}
centroid	What is the centroid of this polygon? Give [x, y].	{"answer": [2.0, 1.5]}
area	What is the area of this polygon?	{"answer": 12.0}
perimeter	What is the perimeter of this polygon?	{"answer": 14.0}
convexity	Is this polygon convex? Answer true or false.	{"answer": true}
orientation	Is the exterior ring of this polygon oriented clockwise or counter-clockwise? Answer "cw" or "ccw".	{"answer": "ccw"}
aspect_ratio	What is the aspect ratio of this polygon's bounding box (width divided by height)?	{"answer": 1.33}
edge_length_variance	What is the variance of the edge lengths of this polygon?	{"answer": 4.21}

Question text for each property.

4 Domain 2: Graphs

4.1 Overview

Undirected graphs serialized as edge lists. The implicit-structure bottleneck in this domain lies in global graph properties (planarity, bipartiteness, diameter) that are not stated in the edge list but are fully determined by it.

4.2 Object Specification

Tier structure. 300 graphs divided into three complexity tiers of 100 graphs each.

Table 6: Graph dataset tiers.

Tier	Count	Nodes	Purpose
Simple	100	6–15	Baseline; small enough for manual verification
Medium	100	16–40	Core measurement
Hard	100	41–80	Stress test

Graph family balance. Within each tier, sample approximately equal numbers of graphs from the following five families. The target is 20 per family per tier, but exact balance is not required; aim for 18–22.

Table 7: Graph generation families.

Family	Generation Method
Erdős–Rényi	<code>nx.erdos_renyi_graph(n, p)</code> , p chosen to produce moderate density (expected degree 3–6).
Barabási–Albert	<code>nx.barabasi_albert_graph(n, m)</code> , $m \in \{2, 3, 4\}$.
Watts–Strogatz	<code>nx.watts_strogatz_graph(n, k, p)</code> , $k \in \{4, 6\}$, $p \in \{0.1, 0.3, 0.5\}$.
Random bipartite	<code>nx.bipartite.random_graph(n1, n2, p)</code> , with $n_1 + n_2 = n$ and $n_1/n_2 \in [0.3, 0.7]$.
Random planar	Generate a random planar graph using <code>nx.random_graphs</code> . or Delaunay triangulation of random points, then randomly remove edges until the desired density is reached.

Why these five families. They produce graphs with diverse structural properties. Bipartite graphs are always bipartite and have chromatic number 2. Planar graphs are always planar. Erdős–Rényi graphs at moderate density are typically non-planar and non-bipartite for $n > 15$. Barabási–Albert graphs have heavy-tailed degree distributions. Watts–Strogatz graphs have high clustering. This diversity ensures that boolean properties like planarity and bipartiteness are not trivially correlated with graph family, and that the dataset contains a meaningful mix of true and false instances for each boolean property.

Validity constraints. Every graph must satisfy:

1. The graph is connected (single connected component).

2. The graph has no self-loops.
3. The graph has no multi-edges.
4. Node count is within the tier’s specified range.
5. The graph has at least $n - 1$ edges (not a tree with trivial structure) and at most $\binom{n}{2}/2$ edges (not too dense to serialize).

Graphs failing any constraint are rejected and regenerated. Fix and record the random seed.

Bipartiteness and planarity balance. The random bipartite family guarantees bipartite instances. The planar family guarantees planar instances. Across the full dataset of 300 graphs, aim for approximately 25–35% bipartite and 25–35% planar. If the other families happen to produce bipartite or planar graphs, record this accurately in the ground truth.

4.3 Serialization Format

Edge list, one edge per line, zero-indexed node labels:

```
GRAPH (n=12, m=18):
0 3
0 7
1 4
1 9
2 5
...
```

The header line states the node count n and edge count m . Each subsequent line contains two space-separated node indices representing an undirected edge. Edges are listed in sorted order (by first node, then second node).

4.4 Properties

Eight properties, classified as local or global.

Table 8: Graph properties.

Property	Type	Locality	Ground Truth	Eval
degree_of_node_0	integer	local	<code>G.degree(0)</code>	exact
edge_count	integer	local	<code>G.number_of_edges()</code>	exact
triangle_count	integer	global	<code>sum(nx.triangles(G).values()) // 3</code>	exact
is_bipartite	boolean	global	<code>nx.is_bipartite(G)</code>	exact
is_planar	boolean	global	<code>nx.check_planarity(G)[0]</code>	exact
diameter	integer	global	<code>nx.diameter(G)</code>	exact
chromatic_number	integer	global	exact computation (see below)	exact
avg_clustering	float	global	<code>nx.average_clustering(G)</code>	rel. error

Chromatic number computation. NetworkX does not provide an exact chromatic number function. Use a brute-force or backtracking coloring algorithm for small graphs ($n \leq 30$). For larger graphs, use the `chromatic_number` function from SageMath, or the greedy upper bound from NetworkX (`nx.coloring.greedy_color`) combined with a certificate: if the greedy coloring uses k colors and the graph contains a k -clique, the chromatic number is exactly k . If no certificate can be obtained, exclude chromatic number from evaluation for that graph and record the exclusion. Do not use approximate values.

Local vs. global classification rationale. Degree of node 0 and edge count can be read directly from the serialization: degree of node 0 is the number of lines containing “0” as an endpoint, and edge count is stated in the header or equals the number of lines after the header. Triangle count, bipartiteness, planarity, diameter, chromatic number, and average clustering all require computation over the full graph structure.

4.5 Prompt Template

You are given an undirected graph as an edge list.

Graph:

{edge_list_string}

Question:

{question_text}

Respond with only a JSON object in this format:

{"answer": <your answer>}

Table 9: Graph question text.

Property	Question	Answer Example
degree_of_node_0	What is the degree of node 0 in this graph?	{"answer": 4}
edge_count	How many edges does this graph have?	{"answer": 18}
triangle_count	How many triangles are in this graph?	{"answer": 7}
is_bipartite	Is this graph bipartite? Answer true or false.	{"answer": true}
is_planar	Is this graph planar? Answer true or false.	{"answer": false}
diameter	What is the diameter of this graph?	{"answer": 5}
chromatic_number	What is the chromatic number of this graph?	{"answer": 3}
avg_clustering	What is the average clustering coefficient of this graph?	{"answer": 0.42}

Question text for each property.

5 Domain 3: Tabular Data

5.1 Overview

Data frames serialized as CSV. The implicit-structure bottleneck here lies in aggregate statistical properties (correlations, monotonicity, functional dependencies) that are determined by the data

but not stated in the CSV.

5.2 Object Specification

Tier structure. 300 tables divided into three complexity tiers of 100 tables each. 250 tables are synthetic with planted properties; 50 tables are derived from real open-license datasets (see below).

Table 10: Tabular dataset tiers.

Tier	Count	Size	Columns	Purpose
Simple	100	8–15 rows	4–6 columns	Baseline
Medium	100	16–30 rows	6–9 columns	Core measurement
Hard	100	31–50 rows	9–12 columns	Stress test

The 50 real-data tables are distributed approximately evenly across tiers (15–20 per tier). They are included to validate that results on synthetic data generalize. The primary analysis uses all 300 tables; a supplementary analysis compares synthetic-only vs. real-only accuracy to check for distribution artifacts.

Synthetic table generation. Each synthetic table is constructed by a controlled generative process that plants specific ground-truth properties.

1. **Column types.** Each table contains a mix of numeric (integer or float), categorical (string with 3–8 distinct values), and optionally one datetime column. At least 2 numeric columns per table.
2. **Planted correlations.** For each table, select 0–3 pairs of numeric columns and plant a linear correlation with a target Pearson r drawn from $\{-0.9, -0.7, -0.5, 0.5, 0.7, 0.9\}$. To plant a correlation of r between columns X and Y : generate $X \sim \mathcal{N}(0, 1)$, generate independent noise $\epsilon \sim \mathcal{N}(0, 1)$, set $Y = rX + \sqrt{1 - r^2}\epsilon$, then rescale both columns to a desired range. Record the planted r as ground truth. The realized sample correlation will differ slightly from the planted value; use the realized sample correlation (computed by `scipy`) as the ground truth, not the planted target.
3. **Planted monotonicity.** For each table, select 0–2 columns and make them monotonically increasing or decreasing. For a monotonically increasing column: generate sorted values with random positive increments. For non-monotonic columns: add perturbations that break monotonicity.
4. **Planted outliers.** For each table, select 0–2 numeric columns and insert 1–3 outliers at positions drawn uniformly at random. An outlier is a value more than 3 standard deviations from the column mean. Record the number and positions of planted outliers.
5. **Planted functional dependencies.** For each table, select 0–1 pairs of columns where column B is a deterministic function of column A (e.g., $B = 2A + 1$ for numeric, or a lookup mapping for categorical). Record the dependency.
6. **Column naming.** Use generic column names: `col_A`, `col_B`, `col_C`, etc. Do not use semantically meaningful names (e.g., “revenue”, “temperature”) to avoid cueing the model with domain knowledge.

Real-data tables. Select 50 tables from open-license datasets (UCI ML Repository, government open data portals, or similar). Requirements:

- License permits redistribution (CC0, CC-BY, or public domain).
- The dataset is not widely used in ML benchmarks (avoid Iris, Titanic, Wine, Boston Housing, Adult Census). The goal is to minimize the chance that the model has memorized the dataset during training.
- Rename all columns to generic labels (`col_A`, `col_B`, etc.) before serialization.
- Subsample rows to fit within the tier’s size range.
- Compute all ground-truth properties empirically on the subsampled table using the same reference implementations as for synthetic tables.
- Manually verify ground-truth properties for at least 10 of the 50 real tables.

Validity constraints. Every table must satisfy:

1. No completely empty columns.
2. No completely empty rows.
3. At least 2 numeric columns.
4. At most 5% null values overall (for synthetic tables, this means planting a small number of nulls deliberately).
5. Row count and column count within the tier’s specified range.

5.3 Serialization Format

Standard CSV with header row:

```
col_A,col_B,col_C,col_D,col_E
12,3.4,high,2024-01-15,7.8
15,2.1,low,2024-02-20,9.1
...
```

Numeric values are formatted with up to 2 decimal places. Categorical values are unquoted unless they contain commas. Null values are represented as empty fields.

5.4 Properties

Eight properties, classified as local or global.

Property query specification. For properties that are column-specific (`column_dtype`, `correlation`, `is_monotonic`, `has_outlier`, `skewness`, `func_dependency`), the question must name the specific column(s). For example: “What is the Pearson correlation between `col_A` and `col_B`?” For each table, query a subset of column-specific properties covering different columns. The total number of property queries per table should be 6–10.

Table 11: Tabular properties.

Property	Type	Locality	Ground Truth	Eval
row_count	integer	local	<code>len(df)</code>	exact
column_dtype	string	local	<code>df[col].dtype</code> mapped to {numeric, categorical, datetime}	exact
null_count	integer	local	<code>df.isnull().sum().sum()</code>	exact
correlation	float	global	<code>df[A].corr(df[B])</code> (Pearson)	rel. error
is_monotonic	boolean	global	<code>df[col].is_monotonic_increasing or _decreasing</code>	exact
has_outlier	boolean	global	any value $> 3\sigma$ from mean	exact
skewness	float	global	<code>df[col].skew()</code> (scipy)	rel. error
func_dependency	boolean	global	see generation procedure	exact

Correlation query design. For each table with planted correlations, query the correlated pairs. Also query at least one pair of uncorrelated columns as a control. The model should report a correlation near zero for uncorrelated pairs and near the planted value for correlated pairs. This lets us measure both true positive (detecting real correlation) and true negative (not hallucinating correlation) performance.

5.5 Prompt Template

You are given a data table in CSV format.

Table:

{csv_string}

Question:

{question_text}

Respond with only a JSON object in this format:

{"answer": <your answer>}

Question text for each property.

6 Evaluation

This section applies uniformly across all three domains.

6.1 Answer Parsing

The parsing pipeline is the same for all domains and all models.

1. Attempt to parse the entire raw output as JSON. If successful, extract the "answer" field.
2. If step 1 fails, search the raw output for the first substring matching the regex `\{.*?\}` (non-greedy). Attempt to parse that substring as JSON.

Table 12: Tabular question text.

Property	Question	Answer Example
row_count	How many rows does this table have? Do not count the header row.	{"answer": 25}
column_dtype	What is the data type of column {col}? Answer "numeric", "categorical", or "datetime".	{"answer": "numeric"}
null_count	How many null or missing values are in this table in total?	{"answer": 3}
correlation	What is the Pearson correlation coefficient between {col_A} and {col_B}?	{"answer": 0.82}
is_monotonic	Is column {col} monotonically increasing or decreasing? Answer true or false.	{"answer": true}
has_outlier	Does column {col} contain any outliers (values more than 3 standard deviations from the mean)? Answer true or false.	{"answer": false}
skewness	What is the skewness of column {col}?	{"answer": -0.45}
func_dependency	Is column {col_B} a deterministic function of column {col_A}? Answer true or false.	{"answer": true}

3. If step 2 fails, apply property-specific fallback:

- Numeric properties: extract the first number (integer or float) from the output.
- Boolean properties: search for {true, false, yes, no} (case-insensitive). Map yes→true, no→false.
- String properties (orientation, column_dtype): search for the expected values ("cw"/"ccw", "numeric"/"categorical"/"datetime").
- List-valued properties (bbox, centroid): search for a pattern matching a list of numbers.

4. If the output reached the max token limit (512 tokens) and no parseable answer was found at any step above, classify as **reasoning_truncated**.

5. If the output did not reach the token limit but no parseable answer was found, classify as **parse_failure**.

6. Both **reasoning_truncated** and **parse_failure** are treated as incorrect.

Parse success threshold. If the parse failure rate (including **reasoning_truncated**) exceeds 15% for any property on any model, investigate the cause before proceeding. Likely causes: the prompt needs rewording for that model, or the max token limit needs to be increased.

6.2 Correctness Criteria

Numeric properties (area, perimeter, centroid, bbox, aspect_ratio, edge_length_variance, correlation, skewness, avg_clustering).

Relative error:

$$\text{rel_error} = \frac{|\hat{v} - v|}{|v|} \quad (1)$$

where v is the ground truth and $\hat{v}$ is the model’s answer. If $|v| < 0.01$, use absolute error $|\hat{v} - v|$ instead to avoid division by near-zero.

Centroid and bounding box use the normalized metrics from the pilot:

$$\text{centroid_error} = \frac{\sqrt{(\hat{x} - x)^2 + (\hat{y} - y)^2}}{d_{\text{bbox}}} \quad (2)$$

$$\text{bbox_error} = \frac{\max(|\hat{x}_{\min} - x_{\min}|, |\hat{y}_{\min} - y_{\min}|, |\hat{x}_{\max} - x_{\max}|, |\hat{y}_{\max} - y_{\max}|)}{d_{\text{bbox}}} \quad (3)$$

Report accuracy at three tolerance levels:

Table 13: Tolerance levels for numeric evaluation.

Tolerance	Threshold	Interpretation
Strict	$\leq 1\%$ relative error	Correct computation
Moderate	$\leq 5\%$ relative error	Approximately correct
Lenient	$\leq 10\%$ relative error	Coarse estimation

Integer properties (vertex_count, row_count, null_count, edge_count, triangle_count, degree_of_node_0, diameter, chromatic_number): exact match.

Boolean properties (convexity, is_bipartite, is_planar, is_monotonic, has_outlier, func_dependency): exact match. Report accuracy, precision, recall, and a 2×2 confusion matrix. For each boolean property, record the base rate (fraction of objects where the property is true) so that trivial-baseline accuracy can be computed.

Categorical properties (orientation, column_dtype): exact match. Report accuracy and confusion matrix.

6.3 Metrics to Report

For every property, domain, model combination, report:

1. Accuracy (at all three tolerance levels for numeric; exact match for others).
2. Accuracy broken down by tier (simple, medium, hard).
3. Accuracy broken down by property locality (local vs. global).
4. For numeric properties: median relative error and 90th percentile relative error, overall and per tier.
5. For boolean properties: precision, recall, and confusion matrix.
6. Parse success rate.
7. Reasoning-truncation rate.

Aggregate metrics.

- **Local-global gap:** for each domain and model, compute the difference between mean local property accuracy and mean global property accuracy. This is the primary signal of the implicit-structure bottleneck.
- **Complexity gradient:** for each property and model, compute the accuracy drop from the simple tier to the hard tier.
- **Cross-model consistency:** for each domain, compute the Spearman rank correlation of per-property accuracies across models. High rank correlation indicates that the difficulty ordering of properties is model-independent.

6.4 Statistical Reporting

All accuracy metrics are reported with 95% bootstrap confidence intervals (10,000 resamples). Comparisons between conditions (e.g., local vs. global, simple vs. hard tier, model A vs. model B) are tested using two-sided bootstrap tests. Report both the effect size (accuracy difference in percentage points) and the p -value.

7 Qualitative Failure Analysis

After computing quantitative metrics for the first model (DeepSeek-V4-Flash), manually inspect a sample of failures to classify them by failure mode. Inspect at least 30 failures per domain (90 total), sampled to cover all tiers and both local and global properties.

Failure taxonomy. The taxonomy below is adapted from the pilot. The same taxonomy applies across all three domains; domain-specific examples are given for illustration.

Failure mode labels are recorded alongside each failed query. After labeling, report the distribution of failure modes per domain and per property locality (local vs. global). This informs the design of mitigation strategies in Experiment 2.

The failure analysis is performed only on the first model (DeepSeek-V4-Flash). It is not repeated for every model. Its purpose is to understand the *types* of errors, not to compare error types across models.

8 Deliverables

At the conclusion of Experiment 1, the following artifacts must exist. The team may choose filenames and directory structure.

1. **Dataset files** (one per domain). Each file contains all objects with their serialized representations and ground-truth properties. Format: JSON.
2. **Dataset summary statistics** (one per domain). Includes count verification, distributional statistics of complexity features (vertex count, node count, row count), and balance statistics for boolean properties. Format: JSON or printed to stdout.
3. **Query scripts** (one per domain or a single domain-agnostic script). Constructs prompts, calls the model API, parses answers, and writes result logs.

Table 14: Failure mode taxonomy (all domains).

Failure Mode	Description and Examples
Parse failure	Output could not be parsed. No usable answer extracted.
Serialization misread	The model misread the serialized structure. <i>Geometry</i> : confused coordinate order, miscounted vertex pairs. <i>Graph</i> : misidentified which nodes are connected. <i>Tabular</i> : misread column alignment, confused header with data.
Arithmetic error	Correct method, wrong numerical computation. <i>Geometry</i> : wrong shoelace sum. <i>Tabular</i> : wrong correlation calculation.
Structural reasoning error	Error in non-arithmetic reasoning about structure. <i>Geometry</i> : wrong convexity determination. <i>Graph</i> : wrong planarity or bipartiteness judgment.
Hallucination	Answer bears no relation to input. A plausible but fabricated number or label.
Instruction failure	Answered a different question than the one asked.
Off-by-one / definition	Answer differs due to definitional disagreement (e.g., counting the closing coordinate).
Reasoning truncated	Output hit token limit during reasoning with no final answer.

4. **Result logs** (one per model per domain). Each log contains one record per query with: object ID, tier, property name, ground truth, raw model output, parsed answer, parse success flag, correctness flag (at each tolerance level), error value (for numeric properties), model name, and timestamp. Format: JSON.
5. **Evaluation script**. Reads result logs, computes all metrics from Section 6, generates summary tables and figures. Outputs per-property accuracy tables, confusion matrices for boolean properties, error distribution histograms, and the local-global gap analysis.
6. **Failure analysis log** (DeepSeek-V4-Flash only). A spreadsheet or JSON file containing the sampled failures with manual failure-mode labels.
7. **Figures**. At minimum:
 - Bar chart of accuracy by property, colored by tier, for each domain and each model.
 - Scatter plot of accuracy vs. object complexity (vertex count, node count, row count) for selected global properties.
 - Heatmap of local-global gap across domains and models.

9 Decision Rules

These decision rules determine how to proceed to Experiment 2 based on Experiment 1 results. They are defined in advance to prevent post-hoc rationalization.

Proceed to Experiment 2 as planned if: In at least two of the three domains, at least two global properties fall below 70% accuracy (at 5% tolerance for numeric, exact match for

boolean/categorical) on the medium or hard tier, while local properties remain above 85%. This is the expected outcome based on the geometry pilot.

Proceed with modified emphasis if: The local-global gap is present but smaller than expected (less than 15 percentage points). In this case, Experiment 2 should focus on property-conditioned and multi-object comparison pipelines (where the gap matters most) and reduce investment in pass-through pipelines.

Reassess if: All properties across all domains achieve above 85% accuracy at 5% tolerance on the hard tier. This would indicate that current-generation models have largely closed the reconstruction gap since the pilot. The project would then shift emphasis to the pipeline compounding experiments, testing whether even small per-step errors compound at longer depths, and to the cross-domain comparison.

Fix harness if: Parse failure rate exceeds 15% for any property on any model. Resolve the parsing issue before interpreting accuracy results.

10 Execution Checklist

1. **Geometry dataset.** Extend the pilot generator from 90 to 300 polygons. Add `aspect_ratio` and `edge_length_variance` to the property set. Verify ground truth on at least 10 polygons (at least 3 per tier).
2. **Graph dataset.** Implement the graph generator with five families. Compute all 8 properties. Verify ground truth on at least 10 graphs (at least 3 per tier). Pay special attention to chromatic number computation; verify against known graphs.
3. **Tabular dataset.** Implement the synthetic table generator with planted properties. Curate 50 real tables. Compute all 8 properties. Verify ground truth on at least 10 tables (5 synthetic, 5 real).
4. **Prompt and parser.** Implement prompt construction and answer parsing for all three domains. Test the parser on a small set of hand-crafted model outputs covering clean JSON, messy JSON, no JSON, and truncated outputs.
5. **DeepSeek-V4-Flash run.** Run all queries for all three domains. Inspect results. Fix any harness bugs.
6. **Failure analysis.** Manually label at least 90 failures (30 per domain) from the DeepSeek-V4-Flash run.
7. **Remaining models.** Run Qwen3-32B, Llama 4 Scout, Gemini 2.5 Flash-Lite, and GPT-4.1 Mini on all queries. Do not modify prompts, parsers, or evaluation logic between models.
8. **DeepSeek-V4-Pro (thinking).** Run on the 20% stratified subsample across all domains.
9. **Evaluation.** Compute all metrics. Generate all tables and figures. Write a summary of findings.